

R Programs Documentation

Table of Contents

1. [Combination Calculator \(nCr\)](#)
 2. [Palindrome Number Checker](#)
 3. [Prime Number Tester](#)
 4. [Statistical Analysis Tool](#)
 5. [String Palindrome Checker](#)
 6. [Strong Number Detector](#)
 7. [SVM Classification \(Basic\)](#)
 8. [SVM Classification \(Modified\)](#)
-

Combination Calculator (nCr)

Purpose

This program calculates the binomial coefficient (n choose r), which represents the number of ways to choose r items from a set of n items without considering the order.

Code

r

 Copy

```
factorial <- function(x) {  
  if (x == 0) return(1)  
  return(prod(1:x))  
}  
nCr <- function(n, r) {  
  return(factorial(n) / (factorial(r) * factorial(n - r)))  
}  
n <- as.integer(readline("Enter value of n: "))  
r <- as.integer(readline("Enter value of r: "))  
result <- nCr(n, r)  
cat("The value of", n, "choose", r, "is:", result, "\n")
```

Algorithm

1. Define a `factorial` function that:
 - Returns 1 if input is 0
 - Otherwise calculates the product of all integers from 1 to x

2. Define an `nCr` function that:
 - Takes two parameters: n and r
 - Calculates $n!/(r!(n-r)!)$ using the factorial function
 3. Get user input for values of n and r
 4. Calculate the combination using the nCr function
 5. Display the result to the user
-

Palindrome Number Checker

Purpose

This program checks if a given number is a palindrome (reads the same forward and backward).

Code

r

 Copy

```
n <- as.integer(readline("Enter the number:"))
rev <- 0
num <- n

while(n>0){
  a <- n %% 10
  rev <- (rev * 10) + a
  n <- n%%10
}

if(rev==num){
  print(paste("Paliandrome"))
}else {
  print(paste("not paliandrome"))
}
```

Algorithm

1. Get a number input from the user
2. Initialize a variable `rev` to 0 and store the original number in `num`
3. Reverse the number:
 - While n > 0:
 - Extract the last digit of n using modulo operation
 - Add this digit to `rev` (multiply current rev by 10 first)

- Remove the last digit from n using integer division

4. Compare the reversed number with the original number

5. Display whether the number is a palindrome or not

Prime Number Tester

Purpose

This program determines if a given number is prime (only divisible by 1 and itself).

Code

r

 Copy

```
n <- as.integer(readline("Enter the number:"))

prime <- function(x){
  if(x<=1){
    return(FALSE)
  }
  for(i in 2:(x-1)){
    if(x%%2==0){
      return(FALSE)
    }
  }
  return(TRUE)
}

if(prime(n)){
  print("THE NUMBER IS PRIME")
}else{
  print("THE NUMBER IS NOT PRIME")
}
```

Algorithm

1. Get a number input from the user
2. Define a `prime` function that:
 - Returns FALSE if the number is less than or equal to 1
 - For each integer from 2 to (x-1):
 - Checks if the number is divisible by 2 (Note: there's a logical error here as it should check if x is divisible by i, not just by 2)
 - Returns FALSE if divisible

- Returns TRUE if no divisors found
3. Call the prime function with the user input
 4. Display whether the number is prime or not

Statistical Analysis Tool

Purpose

This program calculates various statistical measures (minimum, maximum, mean, median, variance, standard deviation, and mode) for numeric columns in the iris dataset.

Code

r

 Copy

```
custom_min <- function(x) min(x, na.rm = TRUE)
custom_max <- function(x) max(x, na.rm = TRUE)
custom_mean <- function(x) mean(x, na.rm = TRUE)
custom_median <- function(x) median(x, na.rm = TRUE)
custom_variance <- function(x) var(x, na.rm = TRUE)
custom_sd <- function(x) sd(x, na.rm = TRUE)
custom_mode <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}
data <- (iris)
numeric_data <- data[sapply(data, is.numeric)]
for (col in colnames(numeric_data)) {
  x <- numeric_data[[col]]

  if (length(x) == 0 || all(is.na(x))) {
    cat("Column:", col, "- No valid numeric data.\n\n")
  } else {
    cat("Column:", col, "\n")
    cat("Min:", custom_min(x), "\n")
    cat("Max:", custom_max(x), "\n")
    cat("Mean:", custom_mean(x), "\n")
    cat("Median:", custom_median(x), "\n")
    cat("Variance:", custom_variance(x), "\n")
    cat("Standard Deviation:", custom_sd(x), "\n")
    cat("Mode:", custom_mode(x), "\n\n")
  }
}
```

Algorithm

1. Define custom statistical functions (all handling NA values):
 - `custom_min`: Returns minimum value
 - `custom_max`: Returns maximum value
 - `custom_mean`: Returns mean value
 - `custom_median`: Returns median value
 - `custom_variance`: Returns variance
 - `custom_sd`: Returns standard deviation
 - `custom_mode`: Returns the most frequently occurring value
 2. Load the iris dataset
 3. Extract only the numeric columns from the dataset
 4. For each numeric column:
 - Check if it contains valid numeric data
 - If no valid data, display a message
 - Otherwise, calculate and display all statistical measures
-

String Palindrome Checker

Purpose

This program checks if a given string is a palindrome (reads the same forward and backward).

Code

r

 Copy

```
a <- function(x) {  
  s <- readline(prompt = "Enter a string:")  
  x <- strsplit(s, "")[[1]]  
  
  if (paste(rev(x), collapse = "") == s)  
  {  
    print(paste(s, "is a paliandrome"))  
  }else {  
    print(paste(s, "is not a paliandrome"))  
  }  
}
```

a()

Algorithm

1. Define a function `is_palindrome` that:
 - Gets a string input from the user
 - Splits the string into individual characters
 - Reverses the character array and joins it back together
 - Compares the reversed string with the original string
 - Displays whether the string is a palindrome or not
 2. Call the function `is_palindrome`
-

Strong Number Detector

Purpose

This program checks if a number is a strong number (the sum of the factorials of its digits equals the number itself, e.g., $145 = 1! + 4! + 5!$).

Code

r

 Copy

```
n <- as.integer(readline("Enter the number:"))
fact <- function(x){
  val <- 1
  for( i in 1:x){
    val <- val*i
  }
  return(val)
}
sum <- 0
temp <- n
while(temp > 0){
  digit = temp %% 10
  sum <- sum + fact(digit)
  temp <- temp %/% 10
}
if(sum == n){
  print("strong")
}else{
  print("not strong")
}
```

Algorithm

1. Get a number input from the user
 2. Define a `fact` function that calculates factorial using a loop
 3. Initialize `sum` to 0 and store the original number in `temp`
 4. While `temp > 0`:
 - Extract the last digit of `temp` using modulo operation
 - Calculate the factorial of this digit and add it to the sum
 - Remove the last digit from `temp` using integer division
 5. Compare the sum with the original number
 6. Display whether the number is a strong number or not
-

SVM Classification (Basic)

Purpose

This program implements a Support Vector Machine (SVM) classifier on the iris dataset using sepal length and width as features, and visualizes the decision boundaries.

Code

r

 Copy

```
library(e1071)
library(ggplot2)
data(iris)
model <- svm(Species ~ Sepal.Length + Sepal.Width, data = iris, kernel = "linear")
x_range <- seq(min(iris$Sepal.Length) - 0.5, max(iris$Sepal.Length) + 0.5, by = 0.05)
y_range <- seq(min(iris$Sepal.Width) - 0.5, max(iris$Sepal.Width) + 0.5, by = 0.05)
grid <- expand.grid(Sepal.Length = x_range, Sepal.Width = y_range)
grid$Species <- predict(model, grid)
p <- ggplot() +
  geom_tile(data = grid, aes(x = Sepal.Length, y = Sepal.Width, fill = Species), alpha = 0.3) +
  geom_point(data = iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species), size = 2) +
  geom_point(data = iris[model$index,], aes(x = Sepal.Length, y = Sepal.Width), color = "black") +
  labs(title = "SVM Decision Boundary with Support Vectors") +
  theme_minimal()
print(p)
```

Algorithm

1. Load required libraries: e1071 (for SVM) and ggplot2 (for visualization)
2. Load the iris dataset

3. Train an SVM model using linear kernel with Species as the response and Sepal.Length and Sepal.Width as predictors
 4. Create a grid of points spanning the feature space
 5. Predict the species class for each point in the grid
 6. Create a visualization using ggplot2:
 - Use colored tiles to represent the decision regions
 - Plot the original data points colored by species
 - Highlight the support vectors with black asterisks
 - Add title and use minimal theme
 7. Display the visualization
-

SVM Classification (Modified)

Purpose

This program implements an improved SVM classifier on the iris dataset with data splitting, radial kernel, and performance evaluation.

Code


```
library(e1071)
library(ggplot2)
library(caTools)

data(iris)

set.seed(123)
split<-sample.split(iris$Species,SplitRatio = 0.8)
train_data<-subset(iris,split==TRUE)
test_data<-subset(iris,split==FALSE)

model<-svm(Species ~ Sepal.Width+Sepal.Length,data=train_data,kernel="radial",
           gamma=0.5,
           cost=1)
prediction<-predict(model,newdata=test_data)
Accuracy<-mean(prediction==test_data$Species)
cat("Accuracy:",Accuracy)

x_range<-seq(min(iris$Sepal.Length),max(iris$Sepal.Length),length.out=200)
y_range<-seq(min(iris$Sepal.Width),max(iris$Sepal.Width),length.out=200)
grid<-expand.grid(Sepal.Length=x_range,Sepal.Width=y_range)
grid$Species<-predict(model,newdata=grid)

p<-ggplot()+
  geom_tile(data=grid,aes(x=Sepal.Length,y=Sepal.Width,fill = Species),alpha=0.3)+
  geom_point(data=train_data,aes(x = Sepal.Length, y = Sepal.Width, color = Species))+
  labs(title = "svm",x="sepal Lenth",y="sepalWidth")
print(p)
```

Algorithm

1. Load required libraries: e1071 (for SVM), ggplot2 (for visualization), and caTools (for data splitting)
2. Load the iris dataset
3. Set a random seed for reproducibility
4. Split the data into training (80%) and testing (20%) sets
5. Train an SVM model using radial kernel with:
 - Species as the response variable
 - Sepal.Width and Sepal.Length as predictors
 - Gamma parameter set to 0.5
 - Cost parameter set to 1

6. Make predictions on the test set
7. Calculate and display the accuracy of the model
8. Create a grid of points spanning the feature space
9. Predict the species class for each point in the grid
10. Create a visualization using ggplot2:
 - Use colored tiles to represent the decision regions
 - Plot the training data points colored by species
 - Add appropriate labels and title
11. Display the visualization